LAPORAN PROJECT

MATA KULIAH PEMROGRAMAN BERORIENTASI OBJEK

APLIKASI PENJUALAN BERBASIS JAVA GUI DAN MARIADB

Disusun oleh:
Yasef Hatam (20240040010)
Yuli Nur Anggraeni (20240040033)

Dosen Pengampu:
Yulham Wahyudin , M.Kom

FAKULTAS TEKNIK, KOMPUTER, DAN DESAIN
PROGRAM STUDI TEKNIK INFORMATIKA
UNIVERSITAS NUSA PUTRA SUKABUMI
2026

BAB I. PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi menuntut proses bisnis, termasuk kegiatan penjualan barang, untuk dikelola secara terkomputerisasi agar lebih efisien, akurat, dan mudah dipantau. Pencatatan transaksi secara manual rentan terhadap kesalahan pencatatan, kehilangan data, dan sulit untuk direkap menjadi laporan. Oleh karena itu, dibutuhkan sebuah aplikasi yang dapat membantu proses pengelolaan data barang, data pelanggan, serta pencatatan transaksi penjualan secara sistematis.

Project ini dikembangkan sebagai pemenuhan tugas mata kuliah Pemrograman Berorientasi Objek (PBO), dengan tujuan menerapkan konsep-konsep PBO secara nyata pada studi kasus aplikasi desktop. Aplikasi dibangun menggunakan bahasa pemrograman Java dengan antarmuka grafis (GUI) berbasis Java Swing, serta MariaDB sebagai sistem manajemen basis data.

1.2 Tujuan

- Menerapkan konsep dasar pemrograman berorientasi objek (class, object, encapsulation, inheritance, polymorphism, dan package) pada studi kasus nyata.
- Mengimplementasikan exception handling untuk menangani kesalahan input maupun kesalahan pada proses akses database.
- Menerapkan konsep basis data lanjutan berupa stored procedure, function, trigger, dan view pada MariaDB.
- Membangun aplikasi desktop yang fungsional untuk mengelola data barang, data pelanggan, data pengguna, transaksi penjualan, dan laporan penjualan.

1.3 Ruang Lingkup Aplikasi

Aplikasi yang dibangun bernama “Aplikasi Penjualan”, mencakup fitur-fitur berikut:

- Login pengguna (autentikasi berbasis username dan password).
- Manajemen data pengguna (CRUD Data User) dengan pembagian peran (role) admin dan kasir.
- Manajemen data barang (CRUD Data Barang) beserta fitur pencarian.
- Manajemen data pelanggan (CRUD Data Pelanggan) beserta fitur pencarian.
- Transaksi penjualan yang mencatat barang yang dibeli, jumlah, serta menghitung subtotal dan total secara otomatis.
- Laporan penjualan yang merangkum seluruh transaksi yang telah terjadi.

BAB II. DESKRIPSI APLIKASI

2.1 Gambaran Umum

Aplikasi Penjualan merupakan aplikasi desktop berbasis Java Swing yang terhubung ke basis data MariaDB melalui JDBC (Java Database Connectivity). Aplikasi ini digunakan oleh pengguna dengan dua peran, yaitu admin dan kasir, untuk mengelola data master (barang, pelanggan, dan pengguna) serta memproses transaksi penjualan hingga menghasilkan laporan.

Alur penggunaan aplikasi dimulai dari halaman Login, dilanjutkan ke halaman Dashboard (Form Utama) yang berisi menu-menu utama, yaitu Master Data (Data Barang, Data Pelanggan, Data User), Transaksi (Transaksi Penjualan), dan Laporan (Laporan Penjualan).

2.2 Arsitektur Aplikasi

Struktur kode aplikasi disusun berdasarkan pemisahan tanggung jawab (separation of concerns) ke dalam beberapa package, sebagai berikut:

- penjualan.model — berisi kelas-kelas entitas (Barang, Pelanggan, User, dan sebagainya) yang merepresentasikan struktur data.
- penjualan.dao — berisi kelas Data Access Object yang bertugas menjalankan operasi basis data (SELECT, INSERT, UPDATE, DELETE, serta pemanggilan stored procedure).
- penjualan.util — berisi kelas utilitas, khususnya kelas koneksi basis data (KoneksiDB) yang menangani koneksi JDBC ke MariaDB.
- penjualan.gui — berisi seluruh form antarmuka pengguna (Login, Dashboard, Data Barang, Data Pelanggan, Data User, Transaksi Penjualan, dan Laporan Penjualan).

Pemisahan ini membuat kode lebih terstruktur, mudah dipelihara, dan setiap package memiliki tanggung jawab yang jelas — sesuai dengan konsep package pada pemrograman berorientasi objek.

2.3 Fitur-Fitur Aplikasi

2.3.1 Login

Pengguna memasukkan username dan password. Sistem akan memverifikasi kecocokan data dengan tabel user di basis data. Jika sesuai, pengguna diarahkan ke halaman Dashboard; jika tidak sesuai, sistem menampilkan pesan kesalahan.

2.3.2 CRUD Data Barang, Pelanggan, dan User

Ketiga modul ini memiliki pola yang serupa: menampilkan data dalam bentuk tabel, menambah data baru, mengubah data yang dipilih, menghapus data, serta melakukan pencarian data (khusus modul Barang dan Pelanggan). Data User memiliki tambahan atribut role untuk membedakan hak akses pengguna.

2.3.3 Transaksi Penjualan

Kasir memilih data pelanggan untuk memulai transaksi baru, kemudian menambahkan barang beserta jumlahnya satu per satu ke dalam transaksi tersebut. Setiap penambahan barang akan otomatis mengurangi stok barang dan menambah total transaksi melalui mekanisme trigger pada basis data, sehingga aplikasi tidak perlu menghitung ulang secara manual di sisi Java.

2.3.4 Laporan Penjualan

Menampilkan rekap seluruh transaksi yang pernah terjadi, meliputi tanggal transaksi, nama pelanggan, kasir yang memproses, dan total transaksi. Data ini diambil dari view yang telah menggabungkan beberapa tabel terkait, sehingga query di sisi aplikasi menjadi sederhana.

BAB III. IMPLEMENTASI KONSEP PEMROGRAMAN BERORIENTASI OBJEK

3.1 Class dan Object

Setiap entitas data pada aplikasi direpresentasikan sebagai sebuah class pada package penjualan.model, misalnya class Barang, Pelanggan, dan User. Saat aplikasi berjalan, data yang diambil dari basis data diubah menjadi object dari class tersebut, sehingga data dapat diolah menggunakan method-method yang telah didefinisikan. Contoh potongan kode class Barang:

```
public class Barang {

    private int idBarang;

    private String namaBarang;

    private double harga;

    private int stok;

    // constructor, getter, dan setter

}
```

3.2 Encapsulation

Seluruh atribut pada class model (idBarang, namaBarang, harga, stok, dan sebagainya) dideklarasikan dengan modifier private, sehingga tidak dapat diakses secara langsung dari luar class. Akses terhadap atribut tersebut hanya dapat dilakukan melalui method getter dan setter yang telah disediakan. Pendekatan ini melindungi integritas data, karena setiap perubahan nilai dapat dikontrol dan divalidasi melalui method setter apabila diperlukan.

3.3 Package

Kode program dikelompokkan ke dalam empat package utama, yaitu penjualan.model, penjualan.dao, penjualan.util, dan penjualan.gui, sebagaimana telah dijelaskan pada Bab II. Pengelompokan ini menerapkan konsep package pada Java, yang bertujuan untuk mengorganisasi kelas-kelas berdasarkan fungsinya serta menghindari konflik penamaan antar kelas.

3.4 Exception Handling

Setiap operasi yang berkomunikasi dengan basis data berpotensi mengalami kegagalan, misalnya koneksi terputus, data tidak ditemukan, atau pelanggaran batasan (constraint) pada tabel. Oleh karena itu, seluruh method pada lapisan DAO mendeklarasikan throws SQLException, dan pemanggilannya pada lapisan GUI dibungkus dalam blok try-catch. Berikut contoh penanganan exception pada proses penambahan data user, yang menangkap kesalahan duplikasi username secara spesifik:

```
try {
```

```
    userDAO.tambah(new User(0, username, password, namaUser, role));
} catch (SQLException e) {
    if (e.getMessage().toLowerCase().contains("duplicate")) {
        // tampilkan pesan: username sudah digunakan
    } else {
        // tampilkan pesan kesalahan database secara umum
    }
}
```

Dengan pendekatan ini, aplikasi tidak akan berhenti secara tiba-tiba (crash) ketika terjadi kesalahan, melainkan menampilkan pesan yang informatif kepada pengguna.

3.5 Inheritance dan Polymorphism

Konsep inheritance dan polymorphism diterapkan pada struktur data pengguna (User), yang memiliki dua peran dengan perilaku berbeda, yaitu Admin dan Kasir. Kelas User bertindak sebagai superclass yang menyediakan atribut umum, sedangkan kelas Admin dan Kasir merupakan subclass yang saling mewarisi atribut tersebut namun memiliki implementasi berbeda pada method yang berkaitan dengan hak akses, sehingga perilaku method tersebut dapat berbeda-beda tergantung jenis objeknya (polymorphism) meskipun dipanggil melalui referensi tipe yang sama.

BAB IV. IMPLEMENTASI BASIS DATA

4.1 Struktur Tabel

Basis data aplikasi diberi nama db_penjualan, terdiri atas lima tabel utama, yaitu user, barang, pelanggan, transaksi, dan detail_transaksi. Tabel transaksi dan detail_transaksi dipisah untuk mengakomodasi satu transaksi yang dapat berisi lebih dari satu jenis barang, dengan relasi one-to-many di antara keduanya.

Tabel	Fungsi	Relasi
user	Menyimpan data akun pengguna aplikasi (admin/kasir)	Direferensikan oleh transaksi
barang	Menyimpan data master barang	Direferensikan oleh detail_transaksi
pelanggan	Menyimpan data master pelanggan	Direferensikan oleh transaksi
transaksi	Menyimpan data header transaksi penjualan	Berelasi ke pelanggan dan user
detail_transaksi	Menyimpan rincian barang per transaksi	Berelasi ke transaksi dan barang

4.2 Function

Function fn_hitung_subtotal digunakan untuk menghitung subtotal (harga dikalikan jumlah) setiap item barang dalam transaksi. Function ini dipanggil dari dalam stored procedure sp_tambah_item_transaksi.

```
CREATE FUNCTION fn_hitung_subtotal(p_harga DECIMAL(10,2), p_jumlah INT)

RETURNS DECIMAL(10,2) DETERMINISTIC

BEGIN

    RETURN p_harga * p_jumlah;

END
```

4.3 Stored Procedure

Stored procedure sp_tambah_item_transaksi menangani proses penambahan satu item barang ke dalam transaksi yang sedang berjalan. Procedure ini memvalidasi ketersediaan barang dan kecukupan stok sebelum menyimpan data, serta dibungkus dalam transaksi basis data (START TRANSACTION ... COMMIT) agar konsisten.

```
CREATE PROCEDURE sp_tambah_item_transaksi(IN p_id_transaksi INT, IN p_id_barang INT, IN
p_jumlah INT)

BEGIN

    -- validasi barang & stok, lalu INSERT ke detail_transaksi

    -- menggunakan SIGNAL untuk menolak transaksi jika stok tidak cukup

END
```

4.4 Trigger

Trigger trg_after_insert_detail dijalankan secara otomatis setiap kali ada baris baru pada tabel detail_transaksi. Trigger ini mengurangi stok barang terkait dan menambah nilai total pada tabel transaksi, sehingga proses tersebut tidak perlu ditulis secara manual pada kode Java.

```
CREATE TRIGGER trg_after_insert_detail

AFTER INSERT ON detail_transaksi

FOR EACH ROW

BEGIN

    UPDATE barang SET stok = stok - NEW.jumlah WHERE id_barang = NEW.id_barang;

    UPDATE transaksi SET total = total + NEW.subtotal WHERE id_transaksi = NEW.id_transaksi;

END
```

4.5 View

View view_laporan_penjualan menggabungkan tabel transaksi, pelanggan, dan user menjadi satu tampilan siap pakai, sehingga kode Java pada modul Laporan Penjualan cukup melakukan SELECT * FROM view_laporan_penjualan tanpa perlu menuliskan ulang proses penggabungan (JOIN) tabel.

```
CREATE VIEW view_laporan_penjualan AS

SELECT t.id_transaksi, t.tanggal, p.nama AS nama_pelanggan,

       u.nama_user AS nama_kasir, t.total

FROM transaksi t

JOIN pelanggan p ON t.id_pelanggan = p.id_pelanggan

JOIN user u ON t.id_user = u.id_user;
```

BAB V. SCREENSHOT APLIKASI

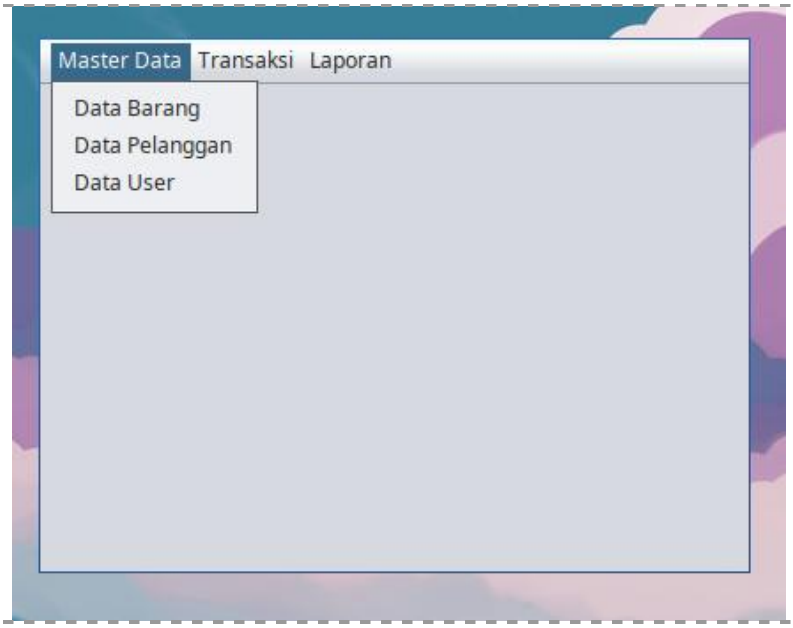
Bagian ini menampilkan tangkapan layar (screenshot) dari aplikasi yang telah dijalankan, sebagai bukti bahwa seluruh fitur berfungsi sebagaimana mestinya.

Gambar 5.1. Halaman Login



Gambar 5.1 — Halaman Login

Gambar 5.2. Halaman Dashboard (Form Utama) beserta menu



Gambar 5.2 — Halaman Dashboard (Form Utama) beserta menu

Gambar 5.3. Halaman Data Barang (CRUD dan pencarian)



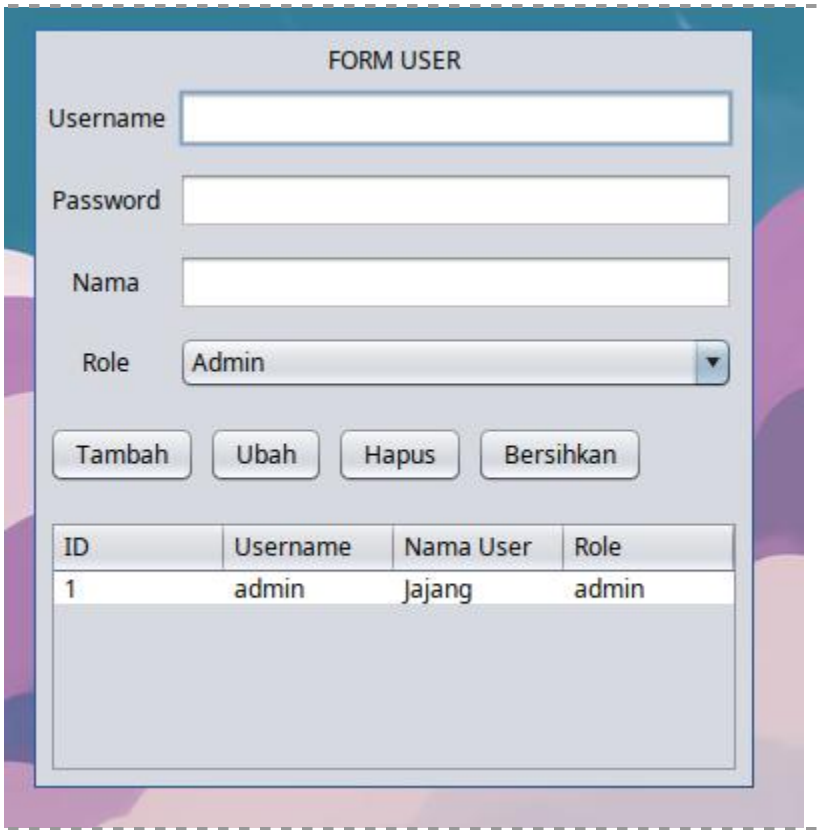
Gambar 5.3 — Halaman Data Barang (CRUD dan pencarian)

Gambar 5.4. Halaman Data Pelanggan (CRUD dan pencarian)



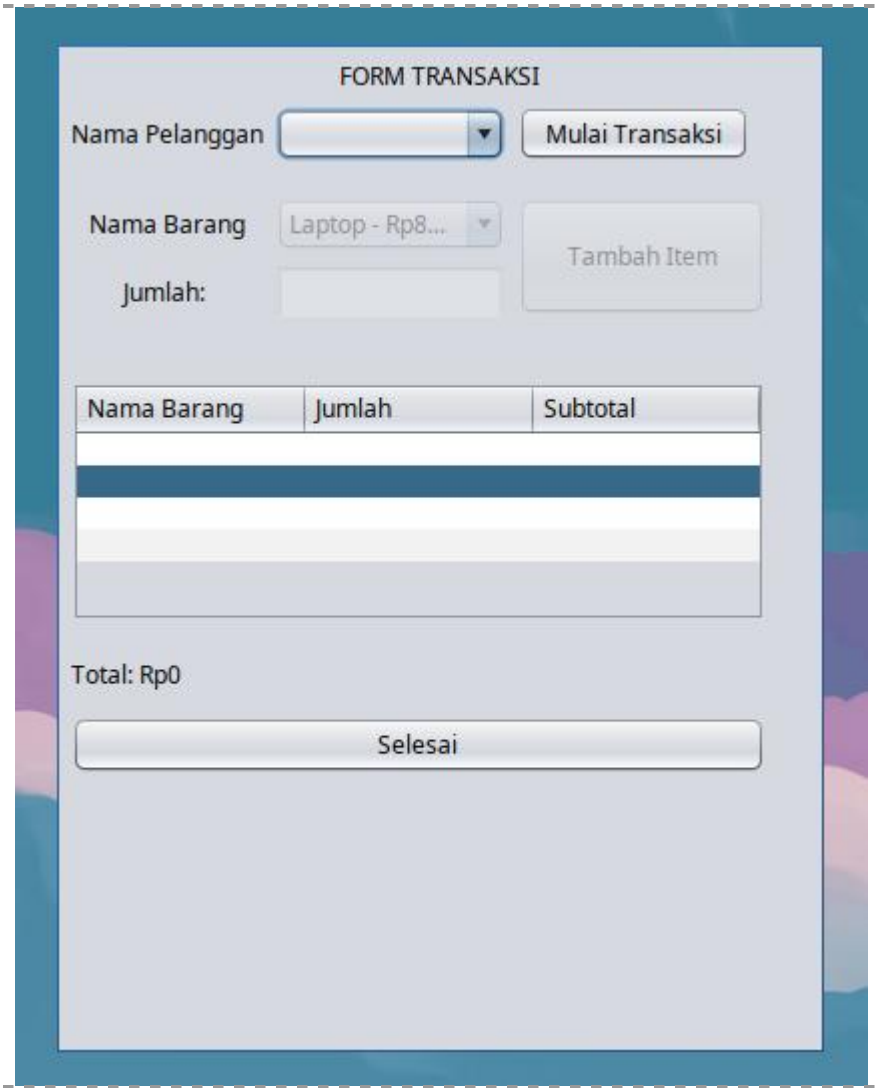
Gambar 5.4 — Halaman Data Pelanggan (CRUD dan pencarian)

Gambar 5.5. Halaman Data User



Gambar 5.5 — Halaman Data User

Gambar 5.6. Halaman Transaksi Penjualan



Gambar 5.6 — Halaman Transaksi Penjualan

Gambar 5.7. Halaman Laporan Penjualan



Gambar 5.7 — Halaman Laporan Penjualan

BAB VI. KESIMPULAN

Berdasarkan proses perancangan dan pengembangan Aplikasi Penjualan Berbasis Java GUI dan MariaDB, dapat disimpulkan beberapa hal sebagai berikut:

- Aplikasi berhasil dikembangkan menggunakan Java Swing sebagai antarmuka dan MariaDB sebagai basis data, dengan seluruh fitur wajib (Login, CRUD Data User/Barang/Pelanggan, pencarian, transaksi penjualan, dan laporan penjualan) berfungsi dengan baik.
- Konsep pemrograman berorientasi objek seperti class, object, encapsulation, inheritance, polymorphism, dan package berhasil diterapkan secara nyata pada struktur kode aplikasi, bukan hanya sebagai teori.
- Exception handling terbukti penting untuk menjaga aplikasi tetap berjalan stabil dan memberikan pesan yang informatif ketika terjadi kesalahan, baik dari sisi input pengguna maupun dari sisi basis data.
- Pemanfaatan fitur basis data lanjutan (stored procedure, function, trigger, dan view) terbukti dapat menyederhanakan kode pada sisi aplikasi Java, sekaligus memusatkan logika bisnis yang kritikal (seperti perhitungan subtotal dan pembaruan stok) pada satu tempat, yaitu basis data, sehingga konsistensi data lebih terjamin.
- Pemisahan struktur kode ke dalam beberapa package (model, dao, util, gui) mempermudah pemeliharaan dan pengembangan aplikasi lebih lanjut di kemudian hari.

Sebagai pengembangan lebih lanjut, aplikasi ini dapat ditambahkan fitur seperti pencetakan struk transaksi, enkripsi password, serta laporan yang dapat difilter berdasarkan rentang tanggal.